Is One Number Enough? Hidden Skill Dimensions in Codeforces Ratings

Vishnu Sinha
CS 328

Bhavay Goyal
CS 328

***Abstract*—Codeforces summarises every contestant with one Elo-style rating. We ask whether that is enough. Using 589 rated rounds (36,519 users, 3,773 problems) we subtract the Elo expectation from every solve and study what is left. The leftover is not noise: it beats two null models, improves prediction of hidden solves, and about six user dimensions reproduce on disjoint halves of the data. The biggest dimension is a difficulty slope (who overperforms on hard problems); the clearest topical one is ad-hoc (math, constructive) versus algorithmic (data structures, graphs, DP). The official tags, however, hardly match the structure users reveal. Finally, MinHash LSH finds "skill twins" while examining 1.4 % of all pairs, but is slower than brute force at this size.**

Code, data and results: https://github.com/BhavayGoyal/cf-skill-dimensions

I. Problem Description

Two users, both rated 1600: one breezes through number theory and freezes on dynamic programming, the other is the opposite. To Codeforces they are the same person. A single rating assumes skill is one-dimensional. We test that assumption, with a test that *can* say "yes, one number is enough":

- Is there structure in who solves what *beyond* rating?
- If so, what are the hidden skills, and do the setters' tags capture them?
- Who are a user's "skill twins"?

II. Solution Approach

A. Survey of Existing Methods

Rating systems. Elo [1], Glicko [2], TrueSkill [3] and Elo-MMR [4] (the family Codeforces builds on) keep one scalar per player; they differ in how they model uncertainty and multi-player rounds, not in dimensionality.

Item response theory. The Rasch model [8] places person ability and item difficulty on one scale; multidimensional IRT [9] gives each person a vector of abilities. Our analysis asks, non-parametrically, how many dimensions the Codeforces problem pool needs.

Educational data mining. Knowledge tracing [6] and Q-matrix/DINA models [7] describe learners by skill mastery over a hand-made item–skill table. The Codeforces tags are such a table; we test it against the data.

Low-rank models. Matrix factorisation [5] is standard in recommenders, and parallel analysis [10] decides how many components are real by comparing with random data.

Similarity search. MinHash [11] with banded LSH [12] finds similar sets without comparing all pairs.

To our knowledge these tools have not been combined to ask whether Codeforces ratings hide skill dimensions, which is what we do below.

B. Our Approach

Data. Public Codeforces API only. We use the 600 most recent finished individual rounds that were at least 90 days old (so problem ratings are settled); 11 were unrated, leaving **589 rounds**. Users need ≥ 40 rated participations: **36,519 users** (median rating 1420), **3,773 problems**, 15.8 M observed user–problem outcomes.

Surprise. For user i (pre-round rating R_i) and problem j (difficulty r_j) Elo [1] predicts a solve probability $p_{ij} = 1/(1+10^{(r_j-R_i)/400})$. The *surprise* is $s_{ij} = y_{ij} - p_{ij}$ with $y_{ij} = 1$ if solved in contest. Solving a hard problem gives a large positive s; missing an easy one a large negative s. Rating is thereby subtracted out; asking how many dimensions remain is a non-parametric version of multidimensional item response theory [8], [9]. Problems a user did not face are 0, giving a sparse $36{,}519 \times 3{,}773$ matrix S.

Is there structure? Three independent tests. (1) A truncated SVD of S is compared with the SVD of copies (the idea of parallel analysis [10]) where each problem's surprises are shuffled among its users, and with a simulation where $y \sim \mathrm{Bernoulli}(p)$, i.e. where rating really is the whole story. (2) We hide 10 % of outcomes and check whether a rank-k model of the rest predicts them better than Elo plus user and problem offsets. (3) We split the rounds into odd and even halves, fit each separately, and check that users get the same scores both times.

What is it? Each axis is read from its most extreme problems and the enrichment of their tags. Problems are also clustered on the axes and compared with the tags, against random clusters of equal size. Users become sets of tokens (`hit`: solved while $p < 0.3$; `miss`: failed while $p > 0.7$); MinHash with banding [11], [12] finds users with similar sets.

III. Results

A. One number is not enough

All three tests agree (Fig. 2); Fig. 3 shows the underlying SVD. The structure survives subtracting every user's per-round average, so it is not just "good day / bad day". A caution: with 15.8 M entries even tiny effects are significant, so all 60 tested components clear the null without being 60 skills. We read the

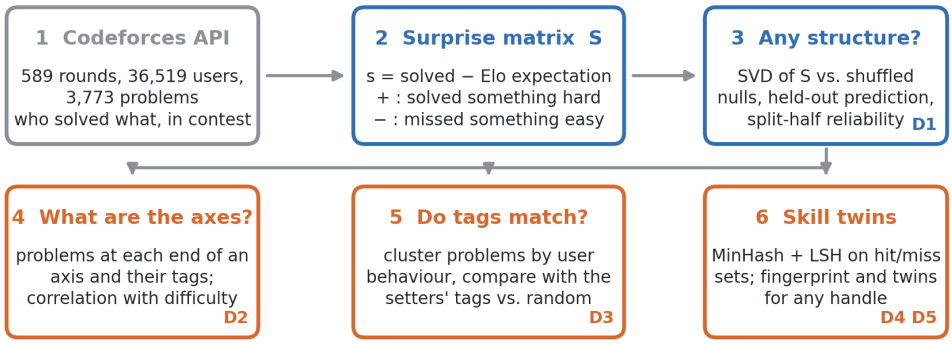


Fig. 1. The pipeline. Blue boxes decide *whether* there is structure; orange boxes describe it. D1–D5 are the project deliverables.

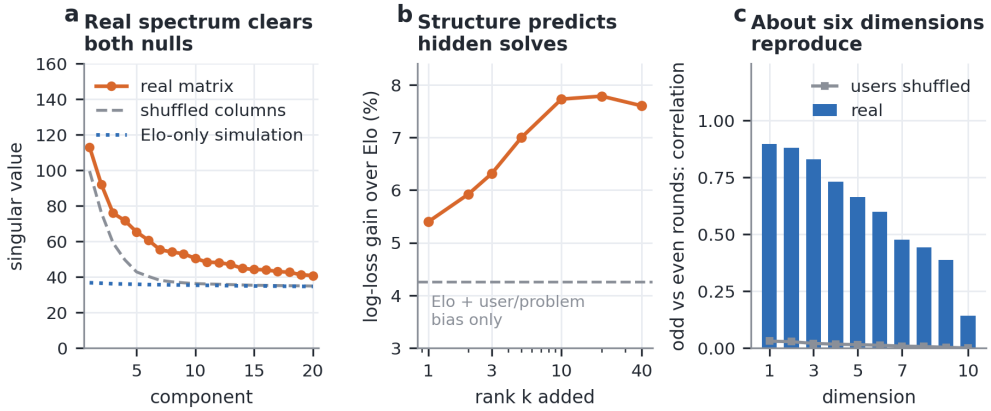


Fig. 2. Three tests that could each have said "no". **a** The real singular values (orange) lie above both nulls; the gap is widest at components 4–9 (up to 1.5×). **b** Hidden-outcome prediction: a rank-10 model improves log-loss by 7.7 % over Elo (0.2135 → 0.1970), 3.6 % beyond Elo plus offsets. It peaks at k=10–20 and then overfits. **c** Odd-round and even-round user scores correlate 0.90 to 0.60 for six dimensions; shuffled users give ≤ 0.03.

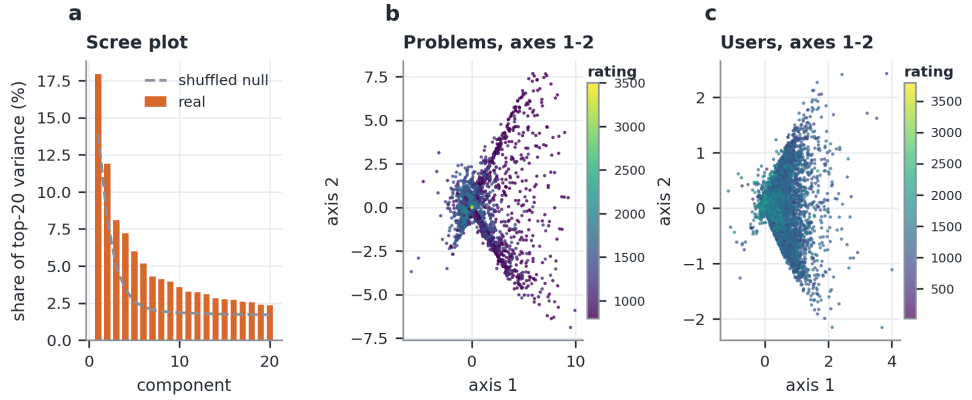


Fig. 3. The SVD itself. **a** Scree plot: share of the top-20 variance per component against the shuffled-column null (dashed); the real spectrum sits above it. **b** Problems projected on the first two axes ($V\Sigma$), coloured by rating: easy problems (dark) fan out along two arms, since a surprise on an easy problem is large. **c** Users on the same axes ($U\Sigma$, 8,000 random users): lower-rated users spread furthest along axis 1, the difficulty slope.

effect sizes, which put the useful content in roughly the first ten dimensions.

B. What the hidden dimensions are

Two patterns stand out (Fig. 4).

- **A difficulty slope (axis 1).** One end is easy implementation problems, the other hard graph/DP/tree problems (correlation −0.47 with problem rating). Users at the hard end over-perform relative to their rating; tourist and jiangly are both at the 12th–13th percentile. Inside a single rating band this axis nearly vanishes (ratio 1.00–1.06), so part of it is a division effect.
- **Ad-hoc versus algorithmic.** Math, constructive, geometry and number theory problems sit opposite data struc-

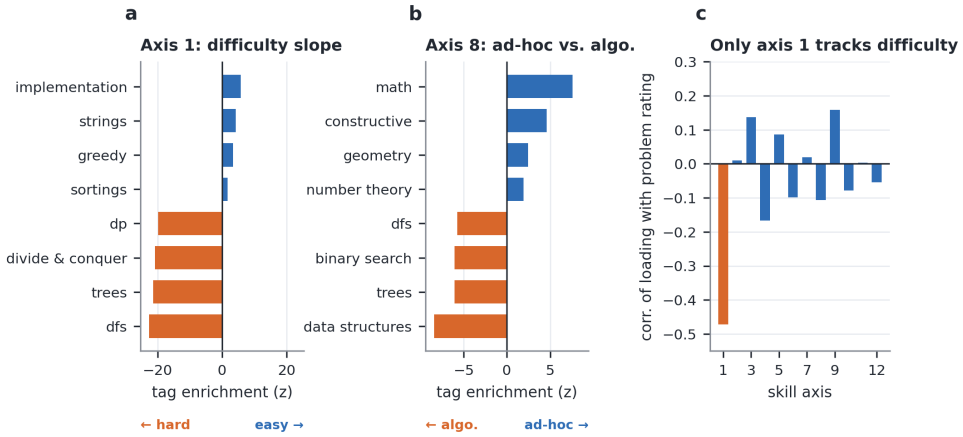


Fig. 4. **a, b** Tags enriched at each end of two axes (z-score of mean loading; signs are arbitrary). **c** Correlation of each axis with problem difficulty: only axis 1 is mostly difficulty.

tures, trees, binary search and graph problems. It appears on axes 2, 3, 4, 8, 11, 12 with $|\rho| \leq 0.17$ against difficulty (axis 8: math $z{=}{+}7.6$, data structures $z{=}{-}8.4$), it is the clearest topic axis inside every rating band, and it survives the per-round correction.

Being good at a tag beyond one's rating is a stable trait mainly for algorithmic tags (split-half r: dp 0.27, data structures 0.26, trees 0.20) and hardly at all for sortings, geometry or brute force (≤ 0.05).

C. Tags only weakly match

The clusters follow difficulty and division, not topic: with $k{=}8$ one cluster holds 2,130 hard problems (mean rating 2035) and the other seven are 800–1400 problems split by position in the round (Fig. 5a). The axes carry information beyond difficulty mainly for ad-hoc tags (math $0.53 \to 0.62$ AUC, constructive $0.52 \to 0.61$, games $0.46 \to 0.59$) and none for dp, trees or graphs, whose tags already reveal their difficulty (Fig. 5b). Tags are related to the hidden dimensions, but do not partition problems the way people behave.

D. Skill twins and LSH

Nearest neighbours are far apart: the median top-1 Jaccard is only 0.12. The 32×4 banding planned in our proposal (threshold 0.42) produces 72 candidate pairs and recall 0.003. With 128×2 banding and 256 hashes LSH reaches recall@10 of 0.84 on all 35,811 users while checking 1.4 % of pairs, but needs 43 s against 12 s for exact brute force (Fig. 6). The saving in pairs is real, the saving in time is not at $n \approx 36$k.

The demo (`python -m cfskills.demo handle`, or the Streamlit app) returns this fingerprint and the closest twins for any handle (Fig. 7). Twins are necessarily close in rating, since two users can share tokens only on problems they both faced.

IV. Conclusions

Limitations. Filling unseen pairs with 0 can let "which division you play" leak into axes. Problem ratings are estimated from solve data (a mild circularity; a per-problem offset gave the largest single prediction gain). Most observations come from 800–1400 problems, only in-contest solves are used, so implementation speed is mixed into axis 1. The LSH implementation skips buckets of over 2,000 users.

Conclusion.

- One number is **not** enough: the residual structure passes a shuffle test, an Elo simulation, held-out prediction and split-half replication.
- Most of it is a **difficulty slope** plus a coarse **ad-hoc versus algorithmic** contrast, not a 35-tag skill vector.
- Official tags are **weakly aligned** with what users actually reveal.
- A rating plus two or three axis scores would describe a contestant better than the rating alone.

Code and Data

All code, the processed surprise matrix and every result file are in one repository: https://github.com/BhavayGoyal/cf-skill-dimensions (package `cfskills`, 21 unit tests, a Makefile that rebuilds every figure and this report; processed data in `data/processed`, results in `outputs`). The raw API cache (118 MB) is not stored; `make fetch` rebuilds it in about 75 minutes.

Contributions

Vishnu Sinha: API pull, surprise matrix, SVD and null tests, D1–D2. *Bhavay Goyal*: MinHash LSH, evaluation, demo, D4–D5. Shared: clustering, tag comparison, robustness checks, report.

References

[1] A. E. Elo, *The Rating of Chessplayers, Past and Present*. Arco, 1978.
[2] M. E. Glickman, "Parameter estimation in large dynamic paired comparison experiments," *J. Royal Statistical Society C*, 48(3):377–394, 1999.
[3] R. Herbrich, T. Minka, T. Graepel, "TrueSkill: A Bayesian skill rating system," in *NIPS 19*, 2007.
[4] A. Ebtekar and P. Liu, "Elo-MMR: A rating system for massive multiplayer competitions," in *Proc. WWW*, 2021.
[5] Y. Koren, R. Bell, C. Volinsky, "Matrix factorization techniques for recommender systems," *IEEE Computer*, 42(8):30–37, 2009.

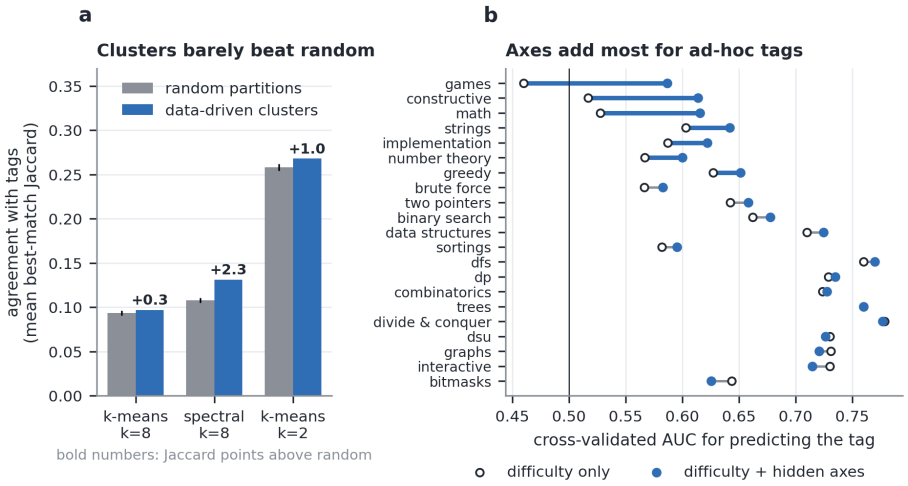


Fig. 5. **a** Clusters of problems built from user behaviour agree with tags only 0.3–2.3 Jaccard points more than random clusters (bars show random ±2 s.d.). **b** Predicting a tag from difficulty alone (hollow) or difficulty plus the 20 axes (filled): axes help mainly for ad-hoc tags.

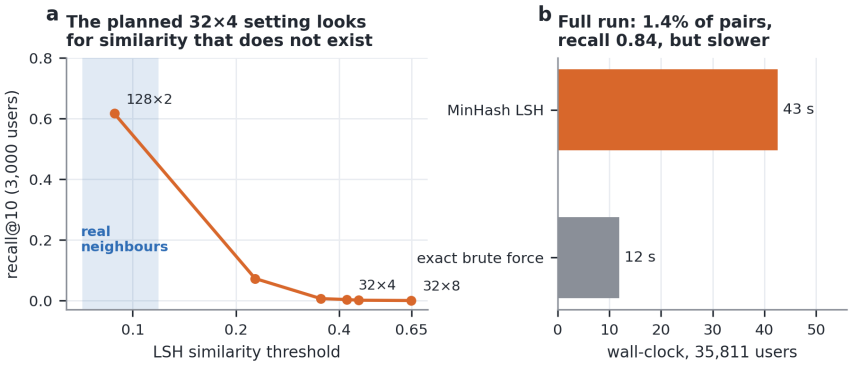


Fig. 6. **a** Recall of the true 10 nearest neighbours against the LSH threshold. Real neighbours are far apart (shaded: Jaccard 0.07–0.12), so only a low threshold finds them. **b** Full population: LSH examines 1.4 % of the 641 M pairs at recall 0.84, yet takes longer than a sparse matrix product.

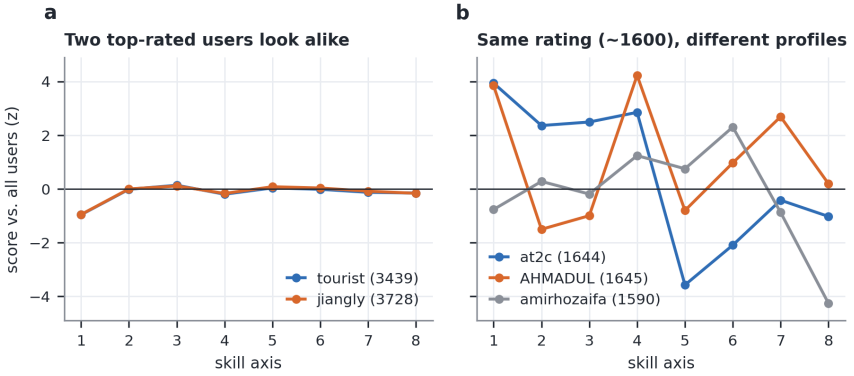


Fig. 7. Skill fingerprints (score on each axis relative to all users). **a** The two top-rated users are almost identical and each other's closest twin (J=0.216). **b** Three users around rating 1600 look very different, which is the point of the project.

[6] A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," *User Modeling and User-Adapted Interaction*, 4:253–278, 1995.

[7] J. de la Torre, "DINA model and parameter estimation: A didactic," *J. Educational and Behavioral Statistics*, 34(1):115–130, 2009.

[8] G. Rasch, *Probabilistic Models for Some Intelligence and Attainment Tests*. Danmarks Pædagogiske Institut, 1960.

[9] M. D. Reckase, *Multidimensional Item Response Theory*. Springer, 2009.

[10] J. L. Horn, "A rationale and test for the number of factors in factor analysis," *Psychometrika*, 30:179–185, 1965.

[11] A. Z. Broder, "On the resemblance and containment of documents," in *Proc. Compression and Complexity of Sequences*, 1997.

[12] J. Leskovec, A. Rajaraman, J. D. Ullman, *Mining of Massive Datasets*, 3rd ed. Cambridge Univ. Press, 2020.